南通大学人工智能与计算机学院

课程名称：汇 编 语 言 程 序 设 计

实验名称：实验 1 汇编语言基本指令

任课教师：鲁　松

实验日期：2026 年 10 月 12 日

学生姓名：林志豪 班级：计 252 学号：2542110155

一、目的及要求

通过第 2、3、4、7、9 题，练习字与双字数据的访问、数组寻址、按位与运算以及寄存器高低字节交换，熟悉 8086 汇编程序的数据段、堆栈段和代码段结构。

掌握使用 MASM 编译、LINK 链接以及 DEBUG 单步执行和查看寄存器、内存的方法，将实际运行结果与预期值进行比较。

本次选做 5 题。程序以完整文字形式列出；运行结果使用实际屏幕截图记录；最后将实验报告导出为 PDF。

二、环境（软、硬件平台）

硬件平台：能够运行 Windows 和 DOSBox 的个人计算机。

软件平台：Windows 操作系统、DOSBox 0.74、Microsoft Macro Assembler 5.00、Microsoft Overlay Linker 3.60、DEBUG 及记事本。

工作目录：D:\Study\My_Project\Assembly-language\lab1。将该目录挂载为 DOSBox 中的 C 盘，在记事本中编辑并保存.asm 源程序。

三、内容及步骤（包括程序流程及说明）

选做题目依次为：第 2 题交换两个字变量，第 3 题拆分双字，第 4 题读取数组第 3 个元素，第 7 题保留最低 3 位，第 9 题交换 AX 的高低字节。

通用操作流程：编辑并保存源程序 → 使用 MASM 生成目标文件 → 使用 LINK 生成可执行文件 → 使用 DEBUG 单步执行 → 观察并截图记录结果。

在 DOSBox 中输入以下命令完成工作目录挂载：

```
mount c "D:\Study\My_Project\Assembly-language\lab1"
c:
```

各题均先装入数据段地址，再执行核心指令。DEBUG 的 t 命令用于单步执行，r 用于查看寄存器，d 用于查看内存。截图在核心指令执行后、退出指令执行前完成，避免退出前设置 AX=4C00h 影响观察。

三、内容及步骤（续）

第 2 题　交换两个字变量

题目及测试数据：数据段定义字变量 num1 和 num2，交换它们的值。初值设置为 num1=12、num2=34（十进制）。

思路：先把 num1、num2 分别读入 AX 和 BX，再将 BX 写入 num1、AX 写入 num2，避免在覆盖变量时丢失原值。

程序流程：初始化 DS → 读取 num1 到 AX → 读取 num2 到 BX → 交叉写回两个变量 → 退出。

完整程序　ex2.asm

```
data segment
    num1 dw 12
    num2 dw 34
data ends

stackseg segment stack
    db 128 dup(0)
stackseg ends

code segment
    assume cs:code, ds:data

start:
    mov ax, data
    mov ds, ax

    mov ax, num1
    mov bx, num2
    mov num1, bx
    mov num2, ax

    mov ax, 4c00h
    int 21h

code ends
end start
```

编译及链接：masm ex2.asm; → link ex2.obj; → debug ex2.exe

调试步骤：DEBUG 中先执行 t 2，再用 d ds:0 1 4 观察初值；执行 t 4 后再次查看这 4 个字节。

预期：num1 由 12 变为 34，num2 由 34 变为 12；内存由 0C 00 22 00 变为 22 00 0C 00。

三、内容及步骤（续）

第 3 题　拆分双字变量

题目及测试数据：数据段定义双字变量 total，将其低 16 位送入 AX，高 16 位送入 DX。测试数据为 12345678h。

思路：8086 按小端方式存储数据，低位字节放在低地址。total 处的一个字为 5678h，total+2 处的一个字为 1234h；使用 WORD PTR 明确按字读取。

程序流程：初始化 DS → 读取 total 低 16 位到 AX → 读取 total 高 16 位到 DX → 退出。

完整程序　ex3.asm

```
data segment
    total dd 12345678h
data ends

stackseg segment stack
    db 128 dup(0)
stackseg ends

code segment
    assume cs:code, ds:data

start:
    mov ax, data
    mov ds, ax

    mov ax, word ptr total
    mov dx, word ptr total+2

    mov ax, 4c00h
    int 21h

code ends
end start
```

编译及链接：masm ex3.asm; → link ex3.obj; → debug ex3.exe

调试步骤：DEBUG 中执行 t 4，再用 r 查看 AX 和 DX，用 d ds:0 l 4 查看双字的 4 个字节。

预期：AX=5678h、DX=1234h；内存中的字节依次为 78 56 34 12。

三、内容及步骤（续）

第 4 题　读取数组第 3 个元素

题目及测试数据：数据段定义 5 个字节的数组 arr，将第 3 个元素（下标 2）复制到 backup。数组初值依次为 10、20、30、40、50，backup 初值为 0。

思路：每个元素占 1 字节，下标 2 对应 arr+2。先将该字节读入 AL，再由 AL 写入 backup。

程序流程：初始化 DS → 读取 arr+2 到 AL → 将 AL 写入 backup → 退出。

完整程序　ex4.asm

```
data segment
    arr db 10, 20, 30, 40, 50
    backup db 0
data ends

stackseg segment stack
    db 128 dup(0)
stackseg ends

code segment
    assume cs:code, ds:data

start:
    mov ax, data
    mov ds, ax

    mov al, arr+2
    mov backup, al

    mov ax, 4c00h
    int 21h

code ends
end start
```

编译及链接：masm ex4.asm; → link ex4.obj; → debug ex4.exe

调试步骤：DEBUG 中先执行 t 2，用 d ds:0 1 6 观察数组及 backup；再执行 t 2 并重新查看 6 个字节。

预期：数组不变，backup 由 0 变为 30（1Eh）；最后一个字节由 00 变为 1E。

三、内容及步骤（续）

第 7 题　保留字变量的最低 3 位

题目及测试数据：将字变量除最低 3 位外的其他位全部清零，测试初值为 FFFFh。程序以 maskValue 表示题目中的 mask，避免与 MASM 关键字重名。

思路：0007h 的二进制为 0000 0000 0000 0111。与其进行 AND 运算后，原数最低 3 位保留，其余 13 位均变为 0。

程序流程：初始化 DS → 对 maskValue 与 0007h 进行按位与 → 将结果保存在原变量中 → 退出。

完整程序　ex7.asm

```
data segment
    maskValue dw 0FFFFh
data ends

stackseg segment stack
    db 128 dup(0)
stackseg ends

code segment
    assume cs:code, ds:data

start:
    mov ax, data
    mov ds, ax

    and maskValue, 0007h

    mov ax, 4c00h
    int 21h

code ends
end start
```

编译及链接：masm ex7.asm;　→　link ex7.obj;　→　debug ex7.exe

调试步骤：DEBUG 中先执行 t 2，用 d ds:0 1 2 观察原值；再执行一次 t 并重新查看这 2 个字节。

预期：maskValue 由 FFFFh 变为 0007h；内存由 FF FF 变为 07 00。

三、内容及步骤（续）

第 9 题　交换 AX 的高低字节

题目及测试数据：交换 AX 的高 8 位 AH 与低 8 位 AL。将 AX 初值设置为 1234h，即 AH=12h、AL=34h。

思路：通过 XCHG AH,AL 直接交换 AH 和 AL。交换后 AH=34h、AL=12h，因此 AX=3412h。

程序流程：初始化 DS → 将 1234h 装入 AX → 交换 AH 与 AL → 退出。

完整程序　ex9.asm

```
data segment
    initialValue dw 1234h
data ends

stackseg segment stack
    db 128 dup(0)
stackseg ends

code segment
    assume cs:code, ds:data

start:
    mov ax, data
    mov ds, ax

    mov ax, initialValue
    xchg ah, al

    mov ax, 4c00h
    int 21h

code ends
end start
```

编译及链接：masm ex9.asm; → link ex9.obj; → debug ex9.exe

调试步骤：DEBUG 中执行 t 3，观察 AX=1234h；再执行一次 t，观察 AX 是否变为 3412h。

预期：AX 由 1234h 变为 3412h。

四、运行结果

第 2 题　交换两个字变量

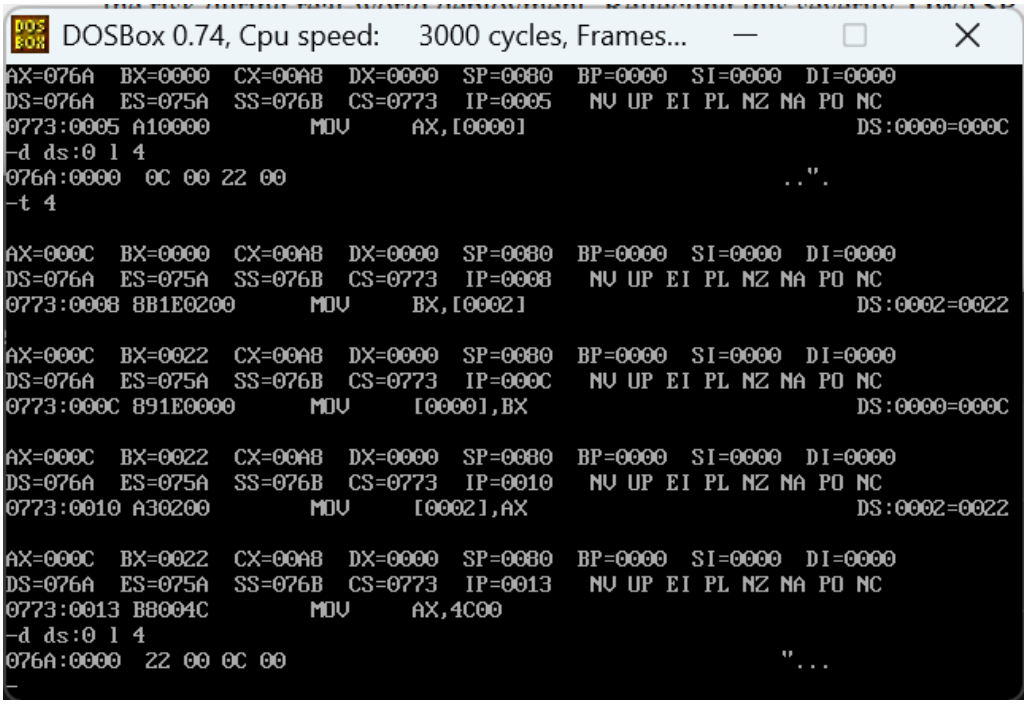


图 1　第 2 题交换前后的内存数据

结果：0C 00 22 00 变为 22 00 0C 00，即 num1=34、num2=12，交换成功。

第 3 题　拆分双字变量

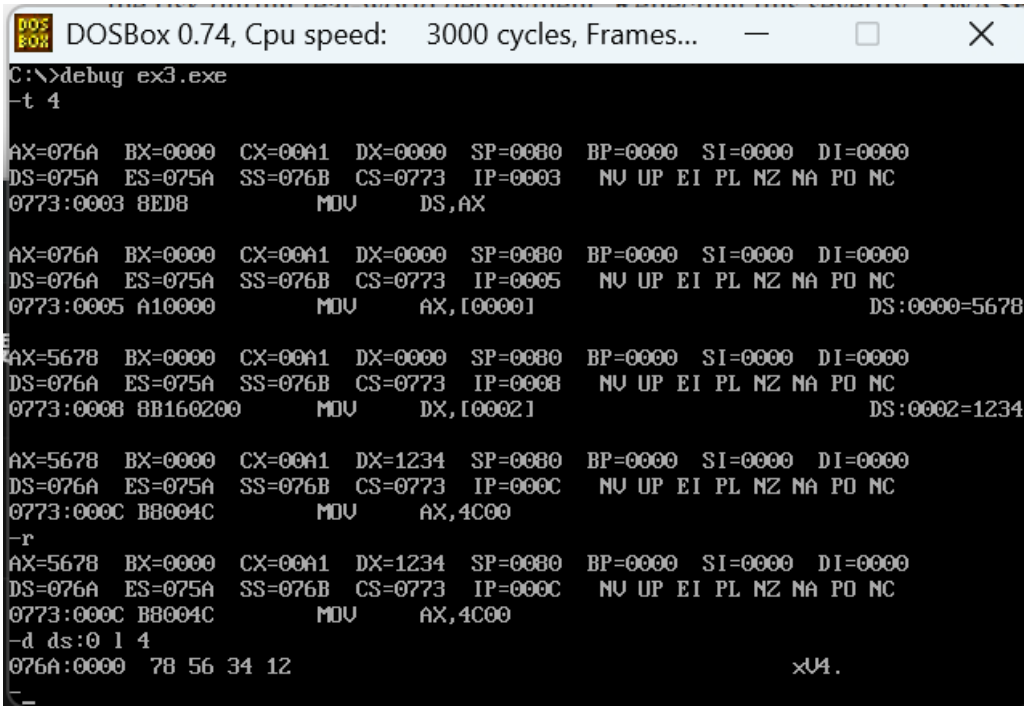


图 2　第 3 题 AX 和 DX 的值及 total 的内存数据

结果：AX=5678h、DX=1234h；内存为 78 56 34 12，低 16 位和高 16 位均读取正确。

四、运行结果（续）

第 4 题　读取数组第 3 个元素

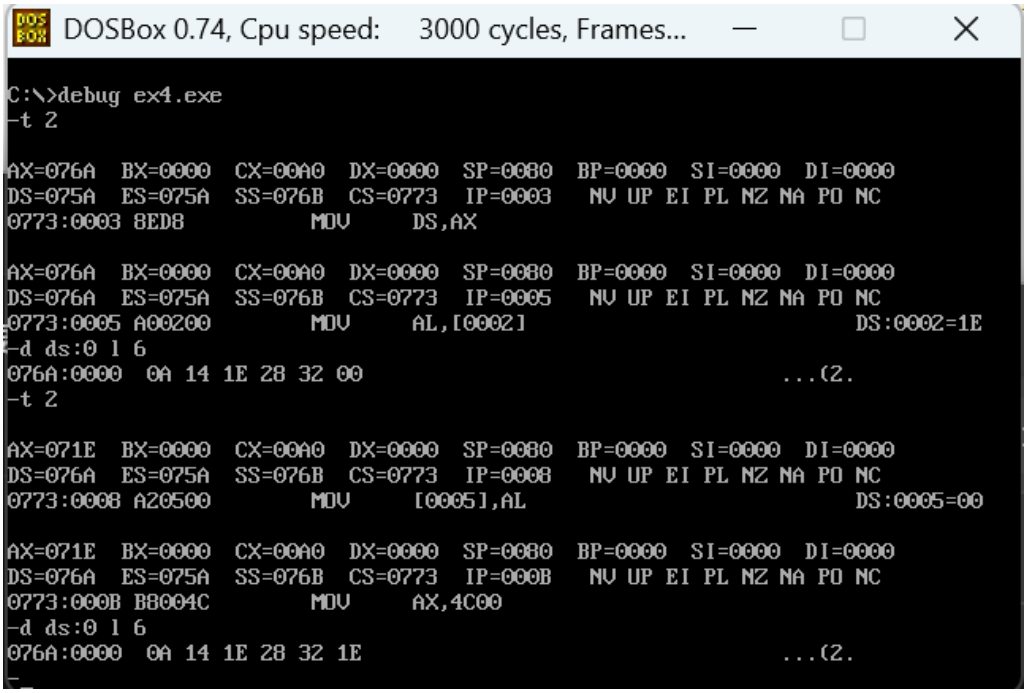


图 3　第 4 题复制前后的数组及 backup 数据

结果：数组前 5 个字节不变，backup 由 00 变为 1E，与第 3 个元素相同，复制成功。

第 7 题　保留最低 3 位

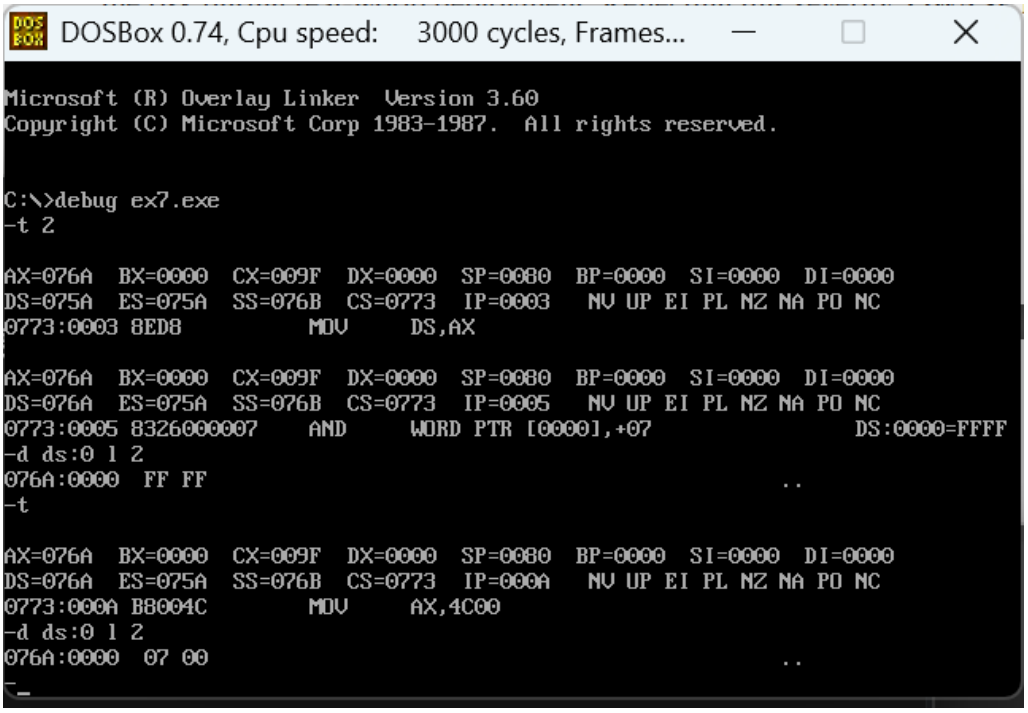


图 4　第 7 题按位与运算前后的数据

结果：FF FF 变为 07 00，即 FFFFh 变为 0007h，最低 3 位保留，其余位清零。

四、运行结果（续）

第 9 题　交换 AX 的高低字节

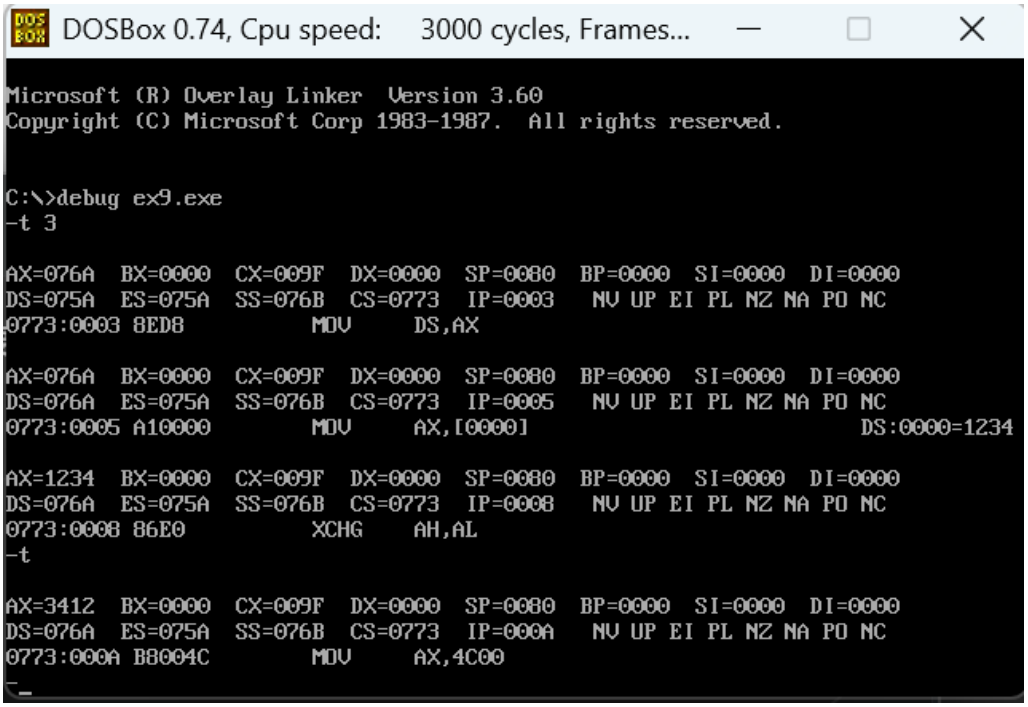


图 5　第 9 题交换前后的 AX 寄存器值

结果：AX 由 1234h 变为 3412h，AH 与 AL 交换成功。五题的实际运行结果均与预期一致。

五、问题及心得

通过本次实验，我初步掌握了从源程序编辑到编译、链接和单步调试的操作过程。观察内存时，要注意 DEBUG 使用十六进制显示，以及字和双字采用小端存储；观察寄存器时，要在核心指令执行后及时记录结果，避免退出程序前设置 AX 的指令覆盖待观察数据。